



UNIVERSITY OF PADOVA

DEPARTMENT OF MATHEMATICS

COURSE IN DATA SCIENCE

OPTIMIZATION METHODS FOR RECOMMENDER SYSTEMS

STUDENTS

REBECCA ESEGIO
GIACOMO FILIPPIN
EMMA LOVATO

IDs

2144564
2130958
2130576

ACADEMIC YEAR

2024-2025

Abstract

This project presents a study on the application of Frank-Wolfe (FW) optimization algorithms and their variants to the Matrix Completion Problem within the context of Recommender Systems. Acknowledging the limitations of traditional optimization methods, particularly the computational cost of projections in high-dimensional settings, we examine the Classic Frank-Wolfe algorithm, highlighting its projection-free nature and $O(1/k)$ sublinear convergence [1]. To address the inherent "zig-zagging phenomenon" that often undermines the Classic FW's practical performance, we analyze two variants: Away-Step Frank-Wolfe (AFW) and Pairwise Frank-Wolfe (PFW) [2]. The project will involve developing efficient implementations of these three algorithms, integrating key components that are crucial for their computational efficiency, such as the Linear Minimization Oracle (LMO), which serves to efficiently find the optimal search direction by minimizing a linear function over the feasible set; adaptive line search techniques, employed to determine optimal step-sizes for robust convergence [3] [1]. These algorithms will then be trained and tested on real-world datasets relevant to Recommender Systems. Through the analysis, including performance plots and tables, we aim to compare the convergence rates and practical effectiveness of Classic FW, AFW, and PFW.

Contents

ABSTRACT	i
2 INTRODUCTION	1
2.1 Introduction to Recommender Systems	1
2.2 Introduction to Frank-Wolfe Algorithms	2
2.3 Problem Statement and Project Objectives	2
3 FORMALIZATION	4
3.1 Problem Formalization	4
3.1.1 The Matrix Completion Problem	4
3.1.2 Convex Relaxation via the Nuclear Norm	5
3.1.3 Components of the Optimization Problem	5
3.2 Step-size Strategies	6
3.2.1 Exact Line Search	6
3.2.2 Diminishing Step-Size	7
3.2.3 Armijo Line Search	7
3.3 Algorithms	8
3.3.1 The Classic Frank-Wolfe Algorithm	8
3.3.2 Away-Step Frank-Wolfe (AFW)	10
3.3.3 Pairwise Frank-Wolfe (PFW)	12
4 METHODS IMPLEMENTATION	14
4.1 Implementation Components	14
4.1.1 Linear Minimization Oracle (LMO)	14
4.1.2 Step-Size Strategies	15
4.2 A Unified Frank-Wolfe Solver	16
4.2.1 Active Set Management	17
5 DATASETS	19
5.1 Data Preprocessing	19
5.1.1 Spectral Accuracy	20
6 RESULTS	21
6.1 Results on MovieLens 100k Dataset	21
6.2 Results on Jester 4 Dataset	23

6.3	Results on Steam Dataset	25
7	CONCLUSIONS	29
7.1	Broader Considerations	29
7.1.1	The Decisive Role of Dataset Properties	29
7.1.2	Step-Size Strategy	31
7.2	Final Conclusion	31
	REFERENCES	32

2

Introduction

2.1 INTRODUCTION TO RECOMMENDER SYSTEMS

Recommender Systems are essential tools that predict user preferences and suggest relevant items, thereby enhancing user engagement and driving business outcomes across various platforms. The core function of these systems, personalizing experiences from movies to products, fundamentally relies on solving optimization problems. These problems typically involve analyzing vast datasets of user-item interactions to train models that can accurately forecast future preferences, with the ultimate goal of minimizing a defined loss function that measures prediction discrepancies.

The optimization problems in Recommender Systems are complex and varied. For example, many popular methods like matrix factorization try to break down large, mostly empty tables of user-item interactions into smaller, easier-to-manage parts. This process needs optimization to reduce errors in recreating the original data. Also, making sure items are shown in the best possible ranking requires advanced optimization methods. A major difficulty for all these methods is scalability; real-world systems deal with massive, sparse datasets, demanding highly efficient algorithms capable of handling both large dimensions and missing data.

Due to these challenges, it is essential to select and implement robust and efficient optimization algorithms for building successful Recommender Systems. This project explores the Classic Frank-Wolfe algorithm and two variants, examining their theoretical principles and practical

applicability to address specific optimization challenges within this domain, with a focus on the Matrix Completion Problem.

2.2 INTRODUCTION TO FRANK-WOLFE ALGORITHMS

The Frank-Wolfe (FW) optimization algorithm, originally introduced by Marguerite Straus-Frank and Philip Wolfe in 1956, has experienced a significant resurgence in recent years, particularly within the fields of Data Science and Machine Learning [3]. This renewed interest is fueled by the growing need for fast, reliable, and projection-free optimization methods that can handle complex, structured constraints commonly found in modern applications.

The core idea of the Frank-Wolfe method is simple yet powerful: instead of relying on computationally expensive projections onto a feasible set, FW approximates the objective function locally with a linear model and then minimizes this linear model over the feasible domain. This linear minimization step is often significantly cheaper than projection. The method iteratively moves towards a minimizer of this linearized objective, making it particularly well-suited for problems where the feasible set admits an efficient Linear Minimization Oracle (LMO) [1].

One of the primary advantages of the Frank-Wolfe algorithm is its "projection-free" nature. Many machine learning problems, especially those involving sparse or low-rank solutions, convex hulls of atomic sets, or matrices with bounded trace norms, can be naturally cast as constrained convex optimization problems. For such problems, computing a projection onto the feasible set can be computationally expensive. FW overcome this by replacing the projection step with a linear optimization step, which often has a much lower computational cost.

The classic Frank-Wolfe algorithm guarantees a sublinear convergence rate of $O(1/k)$, its simplicity and ability to generate sparse or structured solutions have made it a valuable tool [1]. Furthermore, recent advancements have led to the development of several variants, such as Away-Step Frank-Wolfe and Pairwise Frank-Wolfe, which address the limitations of the classic method (like the "zig-zagging phenomenon") and demonstrate improved convergence rates [2].

2.3 PROBLEM STATEMENT AND PROJECT OBJECTIVES

This project is focuses on the application of Frank-Wolfe (FW) optimization algorithms for solving the Matrix Completion Problem within the domain of Recommender Systems. The

Matrix Completion Problem, is a crucial task for predicting missing user-item interactions in sparse datasets. We aim to leverage the projection-free nature and efficiency advantages of FW methods, particularly relevant for the low-rank matrix approximation tasks inherent in recommender systems, where traditional projection operations can be computationally expensive. Specifically, this project will involve the development and testing of three Frank-Wolfe algorithms: Classic Frank-Wolfe (FW), Pairwise Frank-Wolfe (PFW) and Away-Step Frank-Wolfe (AFW).

The primary objective is to evaluate the performance of these algorithms. This will be achieved by training and testing them on three real-world datasets relevant to Recommender Systems. The analysis will involve a thorough comparison of their convergence rates, computational efficiency, and overall effectiveness. The results will be presented and interpreted using plots and tables, providing a clear understanding of their strengths, weaknesses, and the possible performance improvements offered by the variants over the Classic FW algorithm in the context of the Matrix Completion Problem.

3

Formalization

3.1 PROBLEM FORMALIZATION

The Matrix Completion problem is a fundamental task in the field of Recommender Systems. The primary goal is to recover a complete, low-rank user-item interaction matrix from a sparse and incomplete set of observed entries. This section provides the mathematical formalization of the problem that is addressed in this project.

3.1.1 THE MATRIX COMPLETION PROBLEM

Let $U \in \mathbb{R}^{m \times n}$ be the unknown, complete matrix of user-item ratings, where m is the number of users and n is the number of items. We only have access to a small subset of its entries, indexed by the set $\Omega \subset \{1, \dots, m\} \times \{1, \dots, n\}$. The core assumption is that the underlying matrix U is low-rank. The task is to find a matrix $X \in \mathbb{R}^{m \times n}$ that best approximates the known ratings. This can be initially formulated as an optimization problem with a rank constraint:

$$\min_{X \in \mathbb{R}^{m \times n}} \sum_{(i,j) \in \Omega} (X_{ij} - U_{ij})^2 \quad \text{s.t.} \quad \text{rank}(X) \leq \delta \quad (3.1)$$

where δ is a parameter representing the desired maximum rank of the solution matrix [3].

3.1.2 CONVEX RELAXATION VIA THE NUCLEAR NORM

The rank-minimization problem as stated in Equation 3.1 is non-convex and computationally expensive. To address this, the standard approach is to replace the rank constraint with a convex surrogate [3]. The tightest convex relaxation of the rank function is the nuclear norm, denoted by $\|X\|_*$, which is the sum of the singular values of the matrix X [3]. This leads to the following convex optimization problem, which is the focus of this project:

$$\min_{X \in \mathbb{R}^{m \times n}} f(X) := \sum_{(i,j) \in \Omega} (X_{ij} - U_{ij})^2 \quad \text{s.t.} \quad \|X\|_* \leq \tau \quad (3.2)$$

where $\tau > 0$ is a parameter that controls the complexity, or effective rank, of the solution matrix X .

3.1.3 COMPONENTS OF THE OPTIMIZATION PROBLEM

We now formally define the key components of the optimization problem in Equation 3.2 that are relevant to the Frank-Wolfe algorithms.

THE OBJECTIVE FUNCTION AND ITS GRADIENT

The objective function is the squared error over the observed entries:

$$f(X) = \sum_{(i,j) \in \Omega} (X_{ij} - U_{ij})^2$$

This function is convex and continuously differentiable. Its gradient $\nabla f(X)$ is given by:

$$\nabla f(X) = 2(X - U)_\Omega$$

where $(A)_\Omega$ denotes a matrix that is equal to A for indices in Ω and is zero elsewhere. Notably, the gradient matrix is sparse, containing at most $|\Omega|$ non-zero entries. This sparsity is crucial for the computational efficiency of the subsequent algorithmic steps [3].

THE FEASIBLE SET

The feasible set for our problem is the nuclear norm ball of radius τ :

$$C = \{X \in \mathbb{R}^{m \times n} : \|X\|_* \leq \tau\}$$

This set is compact and convex. A critical property for the Frank-Wolfe algorithm is its geometric structure. The nuclear norm ball is the convex hull of all rank-one matrices with a nuclear norm of τ [1] [3]. Formally:

$$C = \text{conv} \{ \tau \mathbf{u} \mathbf{v}^T : \mathbf{u} \in \mathbb{R}^m, \mathbf{v} \in \mathbb{R}^n, \|\mathbf{u}\|_2 = \|\mathbf{v}\|_2 = 1 \}$$

These rank-one matrices, $\tau \mathbf{u} \mathbf{v}^T$, are the fundamental atoms of the feasible set. The Frank-Wolfe algorithm leverages this atomic structure by iteratively adding rank-one matrices to construct a low-rank solution .

3.2 STEP-SIZE STRATEGIES

The step-size, often denoted as α_k or γ_k , is a critical parameter within Frank-Wolfe (FW) algorithms and their variants, profoundly impacting both their convergence characteristics and practical performance. The judicious selection of this step-size is paramount for ensuring the algorithm's stability and optimizing its speed of convergence. Our framework is designed to implement and facilitate a comprehensive comparison of several distinct strategies for determining this crucial parameter, as detailed in the literature.

3.2.1 EXACT LINE SEARCH

Instead of using a pre-determined, open-loop rule, the step-size can be determined dynamically at each iteration by performing an exact line search. This adaptive strategy finds the optimal step-size α_k that minimizes the objective function along the chosen search direction $d^{(k)}$, ensuring maximal progress at each step. The step-size is formally defined as:

$$\alpha_k := \operatorname{argmin}_{\alpha \in [0, \alpha_{max}]} f(x^{(k)} + \alpha d^{(k)})$$

where α_{max} is the maximum allowable step-size that maintains feasibility. This approach is a core component of many advanced Frank-Wolfe variants and is particularly effective when

a closed-form solution for the minimization exists, as is the case for the quadratic objective function in our project [3].

The exact line search is central to the implementation of both the Away-Step and Pairwise Frank-Wolfe variants, where the calculation of α_{max} is crucial for the correctness of the algorithm.

- For the Away-Step Frank-Wolfe (AFW) algorithm, $\alpha_{max} = 1$ for a standard FW step. For an away step, the maximum step-size is set to $\alpha_{max} = \alpha_{v^{(k)}}^{(k)} / (1 - \alpha_{v^{(k)}}^{(k)})$ to ensure the resulting iterate remains a valid convex combination. An away step taken with this maximum value is termed a *drop step* [2].
- For the Pairwise Frank-Wolfe (PFW) algorithm, the maximum step-size is simply the weight of the away atom, $\alpha_{max} = \alpha_{v^{(k)}}^{(k)}$, as no more weight can be removed from $v^{(k)}$ than it currently possesses [2].

3.2.2 DIMINISHING STEP-SIZE

An alternative strategy is the *diminishing step-size*, which employs a pre-defined, open-loop rule. This approach is computationally inexpensive as it does not require any additional calculations. The most common rule is:

$$\alpha_k = \frac{2}{k+2}$$

This specific sequence is a standard choice in the theoretical analysis of the classic Frank-Wolfe algorithm because it is proven to guarantee the $\mathcal{O}(1/k)$ sublinear convergence rate. While the original 1956 paper by Frank and Wolfe relied on a form of line search, this fixed rule became popular in later analyses for its simplicity and robust theoretical guarantees. The primary trade-off is that its non-adaptive nature may lead to slower practical convergence compared to an exact line search [1].

3.2.3 ARMIJO LINE SEARCH

When an exact line search is computationally expensive, the Armijo Line Search provides a robust and practical alternative. This iterative method seeks an acceptable step-size α_k that guarantees a "sufficient decrease" in the objective function. The procedure begins with a trial

step-size, typically α_{max} , and iteratively shrinks it by a fixed backtracking factor $\delta \in (0, 1)$ until the following Armijo condition is satisfied:

$$f(x_k + \alpha d_k) \leq f(x_k) + \gamma \alpha \nabla f(x_k)^\top d_k$$

where $\gamma \in (0, 1/2)$ is the sufficient decrease parameter. This condition ensures that the reduction in the objective function is proportional to both the step-size and the directional derivative, preventing trivial steps while guaranteeing progress [3].

3.3 ALGORITHMS

This chapter details the first-order, projection-free optimization methods implemented and analyzed in this project. We begin with the Classic Frank-Wolfe algorithm and then proceed to its variants, the Away-Step and Pairwise Frank-Wolfe algorithms, which are designed to overcome the classic method's limitations.

3.3.1 THE CLASSIC FRANK-WOLFE ALGORITHM

The Classic Frank-Wolfe (FW) algorithm, also known as the conditional gradient method, is an iterative procedure for solving constrained convex optimization problems of the form

$$\min_{x \in \mathcal{D}} f(x)$$

Its primary advantage lies in its projection-free nature; instead of requiring potentially expensive projections onto the feasible set $\mathcal{D}$, it only needs to solve a linear minimization subproblem over $\mathcal{D}$ at each iteration. This subproblem is handled by a Linear Minimization Oracle (LMO), which makes the algorithm particularly efficient when linear optimization over $\mathcal{D}$ is cheaper than projection, as is the case for the nuclear norm ball in our Matrix Completion problem.

THE FRANK-WOLFE GAP

A key concept in the analysis of the Frank-Wolfe algorithm is the duality gap, often referred to as the FW gap. For a given iterate $x^{(k)}$, the FW gap $g(x^{(k)})$ is defined as:

$$g(x^{(k)}) := \max_{s \in \mathcal{D}} \langle \nabla f(x^{(k)}), x^{(k)} - s \rangle = \langle \nabla f(x^{(k)}), x^{(k)} - s^{(k)} \rangle$$

where $s^{(k)}$ is the solution from the LMO at iteration k . The FW gap provides a computable, worst-case upper bound on the suboptimality of the current iterate, since $g(x^{(k)}) \geq f(x^{(k)}) - f(x^*)$, where x^* is an optimal solution. This makes it an excellent stopping criterion for the algorithm [1].

ALGORITHM DESCRIPTION

The Classic FW method iteratively builds a solution as a convex combination of atoms from the feasible set. The process at each iteration is formally described in Algorithm 3.1.

Algorithm 3.1 The Classic Frank-Wolfe (FW) Algorithm

- o: **Initialize:** Choose an initial feasible point $x^{(0)} \in \mathcal{D}$.
 - o: **for** $k = 0, 1, 2, \dots$
 - o: Find the FW atom $s^{(k)} := \operatorname{argmin}_{s \in \mathcal{D}} \langle s, \nabla f(x^{(k)}) \rangle$. {Linear Minimization Oracle (LMO)}
 - o: Determine the step-size $\gamma_k \in [0, 1]$ using a suitable strategy. {e.g., diminishing rule $\gamma_k = \frac{2}{k+2}$ or exact line search}
 - o: Update the iterate: $x^{(k+1)} := (1 - \gamma_k)x^{(k)} + \gamma_k s^{(k)}$.
 - o: {The update ensures that $x^{(k+1)}$ remains in the convex feasible set $\mathcal{D}$.}
 - o: **end for**
-

CONVERGENCE ANALYSIS

The Classic Frank-Wolfe algorithm is known to exhibit a sublinear convergence rate. Specifically, for a convex and continuously differentiable objective function f with a Lipschitz continuous gradient, and a compact convex feasible set $\mathcal{D}$, the algorithm guarantees that the duality gap (or the error in the objective value) decreases at an $\mathcal{O}(1/k)$ rate [1]. This means that the difference between the current objective value and the optimal objective value,

$$f(x^{(k)}) - f(x^*),$$

converges to zero at the rate $\mathcal{O}(1/k)$. While this is a robust guarantee, it is slower than the linear convergence often achieved by algorithms like gradient descent for unconstrained problems or when specific conditions (e.g., strong convexity) are met.

THE ZIG-ZAGGING PHENOMENON

Despite its simplicity and projection-free nature, the Classic Frank-Wolfe algorithm can suffer from a phenomenon known as zig-zagging or spiking. This behavior typically occurs when the optimal solution x^* lies on the boundary or at a vertex of the feasible set $\mathcal{D}$. In such cases, the LMO step often returns a vertex of $\mathcal{D}$ that is far from the current iterate $x^{(k)}$, but is optimal for the linearized objective. Consequently, the algorithm takes small steps along directions that might not be optimal for global progress, leading to inefficient “zig-zag” movements along the edges of the feasible domain. This oscillation can significantly slow down convergence in practice, even though the theoretical $\mathcal{O}(1/k)$ rate is maintained. This limitation was a key motivation for the development of Frank-Wolfe variants, such as the *Away-Step Frank-Wolfe* and *Pairwise Frank-Wolfe* algorithms, which introduce mechanisms to mitigate this zig-zagging and achieve faster convergence rates [2].

3.3.2 AWAY-STEP FRANK-WOLFE (AFW)

To overcome the limitations of the Classic Frank-Wolfe algorithm, particularly its sublinear convergence rate, the Away-Step Frank-Wolfe (AFW) algorithm was introduced. The core idea is to counteract the zig-zagging phenomenon by allowing for away steps in addition to the standard Frank-Wolfe steps. This provides a mechanism to efficiently remove weight from “bad” vertices in the current solution’s representation, which might otherwise impede progress towards the optimum.

ALGORITHM DESCRIPTION

The Away-Step Frank-Wolfe algorithm enhances the classic framework by maintaining an active set, $\mathcal{S}^{(k)}$, of atoms that constitute the current iterate through a convex combination. At each iteration, the algorithm considers two potential descent directions and greedily chooses the one offering more progress. The complete procedure is formally detailed in Algorithm 3.2 [2].

Algorithm 3.2 The Away-Step Frank-Wolfe (AFW) Algorithm

- o: **Initialize:** Start with an atom $x^{(0)} \in \mathcal{D}$. Initialize the active set $\mathcal{S}^{(0)} = \{x^{(0)}\}$ and its corresponding weight $\alpha_{x^{(0)}}^{(0)} = 1$.
- o: **for** $k = 0, 1, 2, \dots$
- o: Compute the gradient $\nabla f(x^{(k)})$.
- o: Find the FW atom $s^{(k)} := \operatorname{argmin}_{s \in \mathcal{D}} \langle s, \nabla f(x^{(k)}) \rangle$.
- o: Define the FW direction $d_{FW}^{(k)} := s^{(k)} - x^{(k)}$.
- o: Find the away atom $v^{(k)} := \operatorname{argmax}_{v \in \mathcal{S}^{(k)}} \langle v, \nabla f(x^{(k)}) \rangle$ from the active set $\mathcal{S}^{(k)}$.
- o: Define the away direction $d_A^{(k)} := x^{(k)} - v^{(k)}$.
- o: **if** $\langle -\nabla f(x^{(k)}), d_{FW}^{(k)} \rangle \geq \langle -\nabla f(x^{(k)}), d_A^{(k)} \rangle$ {Choose direction with greater progress}
- o: $d^{(k)} \leftarrow d_{FW}^{(k)}$
- o: $\gamma_{max} \leftarrow 1$
- o: **else**
- o: $d^{(k)} \leftarrow d_A^{(k)}$
- o: $\gamma_{max} \leftarrow \alpha_{v^{(k)}}^{(k)} / (1 - \alpha_{v^{(k)}}^{(k)})$
- o: **end if**
- o: Determine the step-size $\gamma_k \in [0, \gamma_{max}]$ (e.g., via exact line search).
- o: Update the iterate: $x^{(k+1)} := x^{(k)} + \gamma_k d^{(k)}$.
- o: Update the active set $\mathcal{S}^{(k+1)}$ and weights $\alpha^{(k+1)}$. {A step with $\gamma_k = \gamma_{max}$ on an away direction is a "drop step"}
- o: **end for**=o

CONVERGENCE PROPERTIES

The key advantage of the Away-Step Frank-Wolfe algorithm is its significantly improved convergence rate. By incorporating away steps, AFW achieves a global linear convergence rate under standard assumptions, such as for strongly convex functions over polytopes. This means the optimality gap decreases geometrically:

$$f(x^{(k+1)}) - f(x^*) \leq (1 - \rho)(f(x^{(k)}) - f(x^*))$$

for some rate $\rho \in (0, 1)$ on every "good step" (i.e., not a drop step). The total number of drop steps is provably bounded, ensuring the overall linear convergence is maintained. This linear rate is a substantial improvement over the $\mathcal{O}(1/k)$ sublinear rate of the classic FW algorithm

and makes AFW a much more efficient algorithm in practice. The convergence rate constant ρ depends on both the function's properties and a geometric property of the feasible set known as the pyramidal width [2].

3.3.3 PAIRWISE FRANK-WOLFE (PFW)

The Pairwise Frank-Wolfe (PFW) algorithm is another powerful variant developed to accelerate the classic Frank-Wolfe method and overcome its slow convergence due to the zig-zagging phenomenon. While AFW chooses between moving towards a new atom or away from an old one, PFW takes a more direct approach: it simultaneously moves weight from the worst atom in the active set to the best new FW atom. This "swap" of weights makes for an aggressive strategy that can be particularly effective at pruning the active set and finding sparse solutions.

THE PFW MECHANISM

The PFW algorithm operates by identifying both a Frank-Wolfe atom $s^{(k)}$ and an away atom $v^{(k)}$ in each iteration. It then combines these into a single *pairwise direction* to update the iterate. The process is detailed in Algorithm 3.3.

Algorithm 3.3 The Pairwise Frank-Wolfe (PFW) Algorithm

- o: **Initialize:** Start with an atom $x^{(0)} \in \mathcal{D}$. Initialize the active set $\mathcal{S}^{(0)} = \{x^{(0)}\}$ and its corresponding weight $\alpha_{x^{(0)}}^{(0)} = 1$.
 - o: **for** $k = 0, 1, 2, \dots$
 - o: Compute the gradient $\nabla f(x^{(k)})$.
 - o: Find the FW atom $s^{(k)} \leftarrow \operatorname{argmin}_{s \in \mathcal{D}} \langle s, \nabla f(x^{(k)}) \rangle$.
 - o: Find the away atom $v^{(k)} \leftarrow \operatorname{argmax}_{v \in \mathcal{S}^{(k)}} \langle v, \nabla f(x^{(k)}) \rangle$ from the active set $\mathcal{S}^{(k)}$.
 - o: Define the pairwise direction: $d^{(k)} \leftarrow s^{(k)} - v^{(k)}$.
 - o: Set the maximum step-size: $\gamma_{max} \leftarrow \alpha_{v^{(k)}}^{(k)}$. {Weight of the away atom}
 - o: Determine the step-size $\gamma_k \in [0, \gamma_{max}]$ (e.g., via exact line search).
 - o: Update the iterate: $x^{(k+1)} \leftarrow x^{(k)} + \gamma_k d^{(k)}$.
 - o: Update the active set $\mathcal{S}^{(k+1)}$ and weights $\alpha^{(k+1)}$. {Transfer weight γ_k from $v^{(k)}$ to $s^{(k)}$ }
 - o: **end for**=o
-

The core of the PFW method is the definition of a single search direction, $d^{(k)} = s^{(k)} - v^{(k)}$, which elegantly combines the addition of a good atom with the removal of a bad one. The

constraint on the maximum step-size, $\gamma_{max} = \alpha_{v^{(k)}}^{(k)}$, is crucial for ensuring that the weights remain non-negative and the iterate $x^{(k+1)}$ remains a valid convex combination [2].

CONVERGENCE PROPERTIES

Similar to AFW, the Pairwise Frank-Wolfe algorithm is proven to achieve a global linear convergence rate under standard assumptions, representing a significant improvement over the classic FW method's $\mathcal{O}(1/k)$ rate. This is because the swap step is also highly effective at preventing zig-zagging.

In practical applications, PFW is often observed to be a very strong performer, sometimes even outperforming AFW. Its direct mechanism of moving weight from the worst to the best atom can be more efficient at reducing the size of the active set. This makes it an excellent choice for problems where the desired solution is expected to be sparse (in our case, low-rank), as it more aggressively seeks a minimal representation for the solution [2].

4

Methods Implementation

This chapter details the practical implementation of the optimization methods evaluated in this project. The algorithms were developed in Python, with a focus on creating a modular and efficient framework. The core logic is divided into two main parts: fundamental components, such as the direction-finding oracle and step-size strategies, and a single, unified solver that orchestrates these components to execute the different Frank-Wolfe variants.

4.1 IMPLEMENTATION COMPONENTS

The efficiency of the Frank-Wolfe algorithms hinges on the performance of their shared components. This section describes the implementation of the Linear Minimization Oracle and the three step-size strategies used in our solver.

4.1.1 LINEAR MINIMIZATION ORACLE (LMO)

At each iteration, the Frank-Wolfe algorithm requires solving a linear subproblem to find the optimal search direction. This is handled by the Linear Minimization Oracle (LMO). For the Matrix Completion problem over the nuclear norm ball, the LMO must find the atom $s^{(k)}$ that minimizes $\langle s, \nabla f(x^{(k)}) \rangle$. The theoretical solution is the rank-one matrix constructed from the top singular vector pair of the negative gradient matrix [3].

This process is implemented in our Python function `linear_minimization_oracle`.

- The function takes the current gradient matrix `grad` and the nuclear norm bound `tau` as input.
- The core of the function is the call to `scipy.sparse.linalg.svds(grad, k=1)`. This function is used to perform an efficient "1-SVD", computing only the single leading singular value and its corresponding left (`u`) and right (`vt`) singular vectors. This is significantly faster than a full SVD, which is critical for performance [1].
- For robustness, the implementation includes a `try...except` block that falls back to the full SVD using `numpy.linalg.svd` in the rare case that the iterative solver fails to converge.
- Finally, the function constructs and returns the rank-one atom s_k using the formula:
`s_k = -tau * (u.reshape(-1, 1) @ vt.reshape(1, -1)).`

4.1.2 STEP-SIZE STRATEGIES

EXACT LINE SEARCH

When the objective function is quadratic, as in our case, the optimal step-size can be calculated analytically. This is implemented in the `exact_line_search` function.

- The function computes the optimal unconstrained step-size γ^* using its closed-form solution:

$$\gamma^* = -\frac{\langle \nabla f(x^{(k)}), P_{\Omega}(d_k) \rangle_F}{2\|P_{\Omega}(d_k)\|_F^2}$$

- It then ensures the final step-size γ_k is clipped to the valid range $[0, \gamma_{max}]$, which is crucial for guaranteeing that the new iterate remains feasible, especially for away and pairwise steps.

DIMINISHING STEP-SIZE

The `diminishing_step_size` function is implemented as follows:

- This function returns the value $\gamma_k = 2/(k + 2)$ based on the current iteration number k .

- This open-loop rule requires no extra computation related to the objective function, making it the fastest per iteration. It provides a robust baseline for convergence with theoretical guarantees.

ARMIJO BACKTRACKING SEARCH

For a more general and robust approach, the `armijo_step_size` function implements a backtracking line search.

- This iterative method guarantees a "sufficient decrease" in the objective function at each step, satisfying the Armijo condition.
- It starts with the maximum allowable step-size, γ_{max} , and reduces it by a multiplicative factor `beta` (e.g., 0.8) until the condition is met.
- This strategy is essential for problems where a closed-form solution for the step-size is not available, and it provides a balance between computational cost and convergence speed.

4.2 A UNIFIED FRANK-WOLFE SOLVER

To facilitate a direct and fair comparison, a single, modular function, `unified_frank_wolfe_solver`, was developed. This function can execute the Classic, Away-Step, or Pairwise variant by specifying a 'variant' parameter. This design consolidates the common steps (gradient calculation, LMO calls) while separating the logic unique to each variant (direction selection, active set management). The complete procedure is detailed in Algorithm 4.1.

Algorithm 4.1 The Unified Frank-Wolfe Solver Implementation

```
o: Inputs: ‘problem’, ‘tau’, ‘variant’, ‘stepsize’, ‘max_iter’, Initialize(0), history, active set  
    $\mathcal{S}^{(0)}$ , and weights  $\alpha^{(0)}$ . {For AFW/PFW, initialize with the first FW atom}  
o: for  $k = 0, 1, 2, \dots, \text{max\_iter} - 1$   
o:   Compute gradient  $\nabla f(x^{(k)})$ .  
o:   Find FW atom  $s^{(k)} \leftarrow \operatorname{argmin}_{s \in \mathcal{D}} \langle s, \nabla f(x^{(k)}) \rangle$ .  
o:   Check FW gap for convergence; if converged, break.  
o:   if ‘variant’ is ‘classic’  
o:      $d^{(k)} \leftarrow s^{(k)} - x^{(k)}$  and  $\gamma_{max} \leftarrow 1$ .  
o:   else  
o:     Find away atom  $v^{(k)} \leftarrow \operatorname{argmax}_{v \in \mathcal{S}^{(k)}} \langle v, \nabla f(x^{(k)}) \rangle$ .  
o:     if ‘variant’ is ‘away_step’  $Choosed^{(k)}$  between  $s^{(k)} - x^{(k)}$  and  $x^{(k)} - v^{(k)}$   
       based on descent.  
o:       Set  $\gamma_{max}$  to 1 or  $\alpha_{v^{(k)}} / (1 - \alpha_{v^{(k)}})$  accordingly.  
o:       ‘variant’ is ‘pairwise’  
o:        $d^{(k)} \leftarrow s^{(k)} - v^{(k)}$  and  $\gamma_{max} \leftarrow \alpha_{v^{(k)}}$ .  
o:       
o:     Determine step-size  $\gamma_k \in [0, \gamma_{max}]$  based on the ‘stepsize’ strategy.  
o:     Update iterate:  $x^{(k+1)} \leftarrow x^{(k)} + \gamma_k d^{(k)}$ .  
o:     if ‘variant’ is ‘away-step’ or ‘pairwise’  
o:       Update active set  $\mathcal{S}^{(k+1)}$  and weights  $\alpha^{(k+1)}$  using helper functions.  
o:     end if  
o:       
o:     return  $x^{(\text{max\_iter})}$  and history. =o
```

4.2.1 ACTIVE SET MANAGEMENT

The AFW and PFW variants depend on explicitly maintaining an active set of atoms and their corresponding weights. This logic is handled by two dedicated helper functions.

- **Away-Step Update:** The `_update_active_set_away_step` function adjusts the weights based on whether a standard FW step or an away step was taken. For an FW step, it adds the new atom $s^{(k)}$ and updates all weights via a convex combination. For an away step,

it increases the weights of all non-away atoms and decreases the weight of the away atom $v^{(k)}$, potentially removing it in a "drop step" [2].

- Pairwise Update: The `_update_active_set_pairwise` function implements the "swap" mechanism. It directly transfers a mass of γ_k from the weight of the away atom $v^{(k)}$ to the weight of the FW atom $s^{(k)}$, adding $s^{(k)}$ to the active set if it is new. This provides a more direct mechanism for pruning the active set [2].

5

Datasets

To evaluate the performance of the Classic Frank-Wolfe algorithms and its variants for matrix completion in Recommendation Systems, three different datasets were used:

- **MovieLens 100k Dataset:** MovieLens data sets were collected by the GroupLens Research Project at the University of Minnesota. This data set consists of: 100,000 ratings from 943 users on 1682 movies.
- **Jester 4 Dataset:** The data is formatted as an excel file representing a 7699 by 159 matrix with rows as users and columns as jokes. The left-most column represents the amount of jokes rated by each user. There are a total of 7699 users and 158 jokes in this dataset.
- **Steam Dataset:** Context Steam is the world's most popular PC Gaming hub, with over 6,000 games and a community of millions of gamers. This dataset consists of: 200,000 interactions (play / purchase) from 12,393 users on 5,155 games.

5.1 DATA PREPROCESSING

For the matrix completion task, the raw data from the three publicly available datasets – MovieLens 100K, Jester 4, and Steam – underwent a series of preprocessing steps to ensure consistency and suitability for the optimization algorithms.

Firstly, all datasets were subjected to a normalization procedure. Ratings (for MovieLens and Jester) and play hours (for Steam) were scaled to the interval $[0, 1]$. This normalization helps in stabilizing the optimization process and ensures that all values contribute equally to the loss function, irrespective of their original scale.

Following normalization, each dataset was divided into training and testing sets. This split is crucial for evaluating the generalization performance of the implemented Frank-Wolfe variants. Specifically, a 60% portion of the observed entries was allocated for the training set, while the remaining 40% was reserved for the testing set. This division ensures a robust evaluation of the algorithms' ability to generalize to unseen data.

Finally, to manage computational complexity and allow for controlled experimentation, the dimensionality of each dataset was reduced. This was achieved by sub-sampling a specific percentage of both users (rows) and items (columns) from the original datasets. This allows for testing the algorithms on smaller, more manageable matrices while still retaining the essential characteristics of the original data structure.

5.1.1 SPECTRAL ACCURACY

The Spectral Accuracy function quantifies the "magnitude" of the unexplained error from the matrix completion model using a metric rooted in singular value decomposition. This metric offers a distinct perspective on model performance compared to traditional error measures like Mean Squared Error (MSE) or Root Mean Squared Error (RMSE).

The largest singular value ($\sigma_{\max}$) of a matrix is equivalent to its spectral norm (or 2-norm), defined as $\|A\|_2 = \sup_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2}$. In the context of matrix completion:

- A high 'accuracy spectral' value indicates that the 'residual' matrix possesses a large spectral norm. This signifies a substantial error in the observed positions. A high value suggests that there exists a direction in the data space (corresponding to the first singular vector) along which the model exhibits a considerable discrepancy with the true observed values. It captures the "maximum deviation" of the model from the real data, rather than an average error.
- Conversely, a low 'accuracy spectral' value (approaching zero) implies a "small" 'residual' matrix, signifying that the model has made accurate predictions for the observed data, minimizing the maximum error across any direction.

6

Results

6.1 RESULTS ON MOVIELENS 100K DATASET

On the MovieLens 100k dataset, the convergence plot illustrates that the Away-Step FW (Exact LS) and Pairwise FW (Exact LS) variants demonstrated the fastest and most stable convergence of the objective function, achieving the lowest final values within 400 iterations. This indicates their efficiency in minimizing the reconstruction error, as they can take more effective descent directions by moving mass away from poor atoms in the active set. The Armijo Line Search strategies for AFW and PFW also performed very well, mirroring the Exact LS results and confirming their reliability.

In contrast, Classic FW variants, regardless of the step-size strategy, showed a slower, more gradual decrease. This is consistent with their known sublinear convergence rate, which can be particularly slow when the optimal solution lies on the boundary of the feasible region, forcing the algorithm to take very small steps. The "diminishing" step-size, while computationally cheap, notably hindered the convergence speed for all variants compared to the more adaptive line search methods, as it lacks the ability to take larger steps when a good descent direction is found.

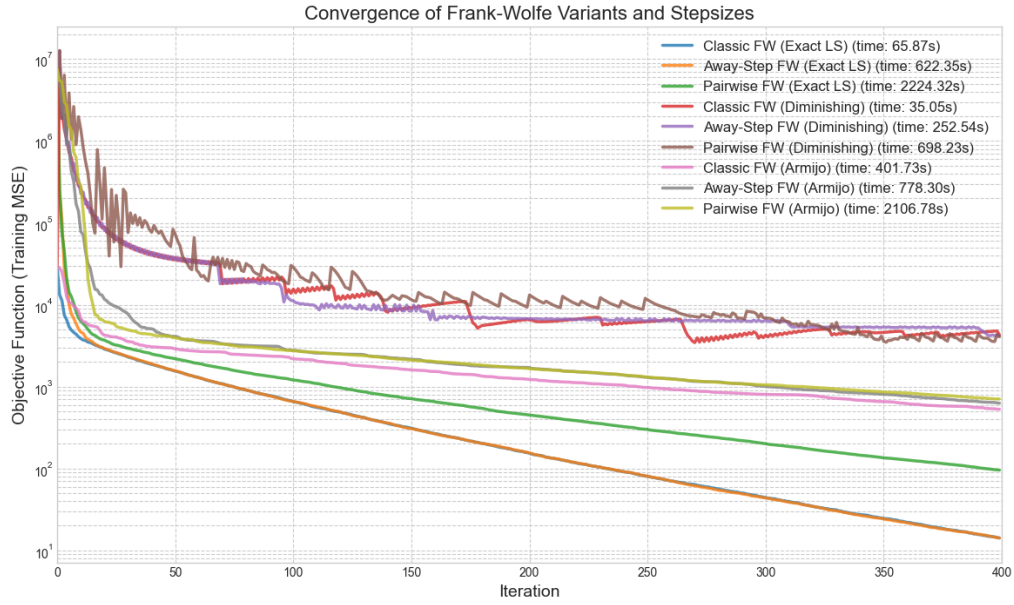


Figure 6.1: Convergence of Frank-Wolfe variants and Stepsizes on MovieLens 100k

The following table summarizes the performance of the three Frank-Wolfe (FW) algorithm variants, combined with three different step-size strategies, on the MovieLens dataset.

Algorithm	Spectral Accuracy	Test RMSE	Time (s)
Classic FW (Exact LS)	9.3989	0.2682	66.6057
Away-Step FW (Exact LS)	11.1992	0.2751	558.8990
Pairwise FW (Exact LS)	6.3981	0.2363	1839.8189
Classic FW (Diminishing)	22.1099	0.3084	31.9902
Away-Step FW (Diminishing)	13.7145	0.2930	205.9545
Pairwise FW (Diminishing)	25.8459	0.3094	633.6213
Classic FW (Armijo)	8.5137	0.2531	386.7354
Away-Step FW (Armijo)	12.6376	0.2682	781.0189
Pairwise FW (Armijo)	7.3684	0.2451	1628.5450

Table 6.1: Experiment Summary on MovieLens Dataset

The Pairwise FW algorithm with Exact Line Search achieved the lowest Test RMSE of 0.2363, signifying the best predictive accuracy on unseen data. This superior generalization, however, comes at a significant computational cost, with this variant being the slowest at over 1800 sec-

onds. This is an expected trade-off: Pairwise FW performs the most work per iteration by simultaneously adding a new atom and removing a poor one, which leads to better convergence and accuracy but requires more computational cost.

Conversely, the Classic FW with Diminishing step-size proved to be the most computationally efficient, completing in just under 32 seconds. This speed comes at the expense of predictive accuracy, as it yielded one of the highest Test RMSE values (0.3084). This highlights its primary weakness: the non-adaptive step size prevents it from effectively minimizing the objective function, leading to poorer generalization.

Regarding Spectral Accuracy, the Pairwise FW (Exact LS) again recorded the best value at 6.3981, so it can be noticed that the algorithm that generalized best (lowest RMSE) also produced the solution matrix that was spectrally closest to the ground truth. This suggests that its update mechanism is not just minimizing error on the observed ratings but is also successfully capturing the underlying structure of the preference data. In contrast, the diminishing step-size variants generally exhibited the worst spectral accuracy values, indicating that their simplicity comes at the cost of learning a structurally accurate solution.

6.2 RESULTS ON JESTER 4 DATASET

On the Jester Dataset, the convergence patterns shown in the plot are strongly validated by the Final Training MSE values. The variants using Exact Line Search or Armijo Line Search demonstrate a powerful and effective convergence, rapidly reaching very low objective function values (e.g., a training MSE of just 0.006 for Pairwise FW with Exact LS). The denser nature of the Jester dataset likely facilitates this progress, allowing the more sophisticated algorithms to find good descent directions efficiently.

In contrast, Classic FW still exhibits its characteristic sublinear progression. The "diminishing" step-size strategy, in particular, leads to extremely poor training performance, with Final Training MSE values in the hundreds (e.g., 389 for Classic FW). This shows that the algorithm barely minimizes the objective function, which is visually represented in the plot by its flat, non-converging trajectory.

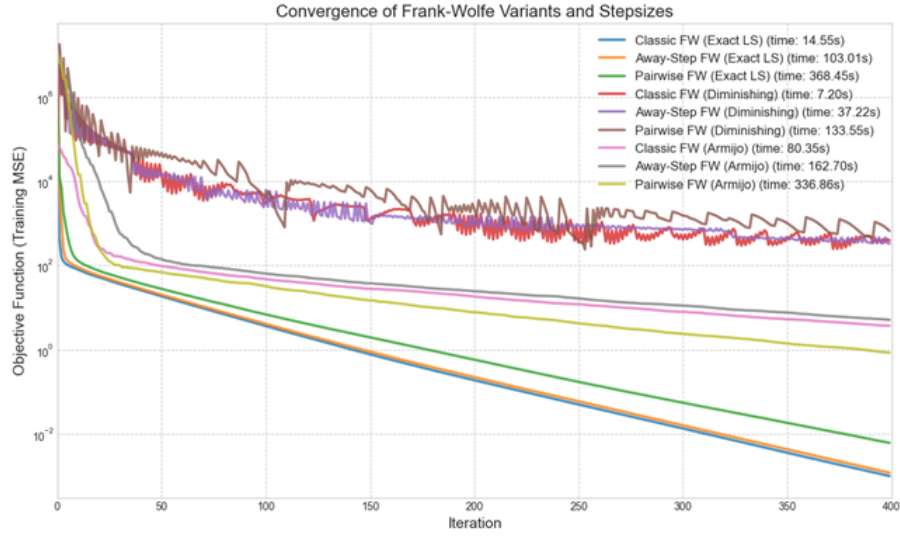


Figure 6.2: Convergence of Frank-Wolfe variants and Stepsizes on Jester 4 Dataset

The following table summarizes the performance of the three Frank-Wolfe (FW) algorithm variants, combined with three different step-size strategies, on the Jester 4 dataset.

Algorithm	Spectral Accuracy	Test RMSE	Time (s)
Classic FW (Exact LS)	2.6491	0.0295	14.55
Away-Step FW (Exact LS)	2.6439	0.0297	103.01
Pairwise FW (Exact LS)	2.6575	0.0265	368.45
Classic FW (Diminishing)	30.6451	0.0944	7.20
Away-Step FW (Diminishing)	20.3442	0.0867	37.22
Pairwise FW (Diminishing)	13.4147	0.0530	133.55
Classic FW (Armijo)	7.0817	0.0544	80.35
Away-Step FW (Armijo)	19.4362	0.0821	162.70
Pairwise FW (Armijo)	2.6692	0.0294	336.86

Table 6.2: Experiment Summary on Jester 4 Dataset

The Pairwise FW algorithm with Exact Line Search demonstrated the best predictive performance, achieving the lowest Test RMSE of 0.0265. This superior accuracy comes at a high computational cost (368.45 seconds), a trade-off consistent with its more complex update rule. In contrast, the Classic FW with Diminishing step-size was the fastest algorithm, requiring

only 7.20 seconds, but this speed came at the cost of poor generalization, resulting in a high Test RMSE of 0.0944.

These results highlights a strong dependence between Test RMSE and Spectral Accuracy for the top-performing models. The three variants with the lowest Test RMSE , Pairwise-Exact LS, Classic - Exact LS, and Pairwise - Armijo, also have the best and tightly clustered Spectral Accuracy values (all around 2.6). This indicates that the models that generalize best are also the ones that most accurately capture the underlying spectral structure of the data matrix. The variants with poor generalization, such as those using the diminishing step-size, show significantly worse spectral accuracy, confirming that they fail to learn the data’s structure. This dataset shows that a proper, adaptive step-size strategy is critical for achieving good performance, with the implicit regularization of a diminishing step-size being an ineffective strategy here.

6.3 RESULTS ON STEAM DATASET

Based on the expected behavior of these algorithms on a sparse dataset like Steam, one would observe that the Away-Step FW and Pairwise FW variants, particularly when combined with Exact or Armijo Line Search, should demonstrate the most rapid and consistent decrease in the objective function, reaching the lowest MSE values within the 400 iterations. This illustrates their superior convergence properties. Conversely, Classic FW variants, irrespective of the step-size strategy, would typically show slower, sublinear convergence, plateauing at higher MSE values. Furthermore, the Diminishing step-size generally leads to slower objective function convergence across all variants compared to the adaptive line search methods, especially for Away-Step and Pairwise FW where it is not the intended step-size. The times noted in the legend would provide additional context, indicating the wall-clock performance trade-offs; while some algorithms converge faster in terms of iterations, their per-iteration computational cost might result in longer total execution times.

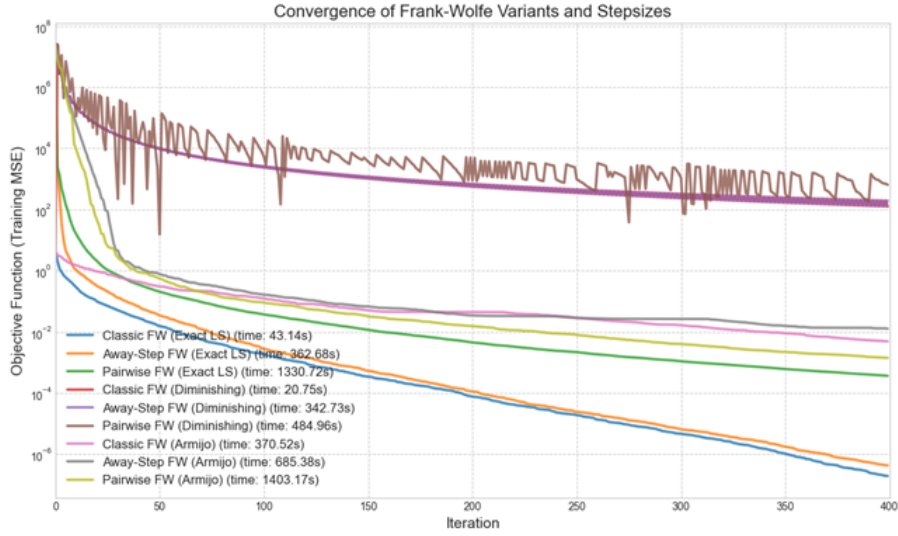


Figure 6.3: Convergence of Frank-Wolfe variants and Stepsizes on Steam Dataset.

The following table summarizes the performance of the three Frank-Wolfe (FW) algorithm variants, combined with three different step-size strategies, on the Steam dataset.

Algorithm	Spectral Accuracy	Test RMSE	Time (s)
Classic FW (Exact LS)	1.5257	0.0405	43.14
Away-Step FW (Exact LS)	15.2910	0.3779	362.68
Pairwise FW (Exact LS)	13.6530	0.3420	1330.72
Classic FW (Diminishing)	1.5251	0.0405	20.75
Away-Step FW (Diminishing)	1.5252	0.0405	342.73
Pairwise FW (Diminishing)	6.0816	0.1360	484.96
Classic FW (Armijo)	1.5109	0.0431	370.52
Away-Step FW (Armijo)	1.4017	0.0388	685.38
Pairwise FW (Armijo)	3.2669	0.0739	1403.17

Table 6.3: Experiment Summary on Steam Dataset

Based on the Test RMSE, which is the most critical metric for generalization performance, the Away-Step FW with Armijo step-size emerges as the top-performing algorithm, achieving the lowest error of 0.0388.

However, a crucial trade-off between performance and efficiency is evident. The Classic FW with a Diminishing step-size strategy achieves a Test RMSE of 0.0405, which is almost identical

to the performance of the much more complex Exact Line Search variant (0.0405), but it does so in just 20.75 seconds, the fastest time recorded. The Pairwise FW variants are consistently the slowest, with execution times exceeding 1300 seconds for Exact LS and Armijo. This computational cost, combined with sub-optimal Test RMSE in this experiment, makes them less practical for this specific problem.

An analysis of the training and test errors reveals two distinct behaviors across the different step-size strategies, highlighting the concepts of overfitting and implicit regularization.

The most striking result is observed with the Diminishing step-size variants. These algorithms exhibit a very high Final Training MSE (e.g., 124.79 for Classic FW), indicating that they are underfitting the training data. The models are not learning the training set well at all.

However, this poor performance on the training data leads to excellent performance on the unseen test data. The Classic and Away-Step FW with a Diminishing step-size achieve a Test RMSE of 0.0405, which is among the best results. This phenomenon occurs because the diminishing step-size schedule becomes very small, very quickly. It effectively stops the optimization process early, preventing the model from fitting the noise and specific artifacts present in the training data. This behavior acts as a strong form of implicit regularization, forcing the model to only learn the most general patterns, which leads to better generalization.

In contrast, the algorithms using Exact Line Search and, to a lesser extent, the Armijo rule, show clear evidence of overfitting.

Overfitting is defined by a significant gap between performance on the training data and performance on the test data. For instance, the Classic FW (Exact LS) achieves a very low Final Training MSE of 1.94×10^{-7} . This demonstrates that the model has almost perfectly memorized the training set. However, its performance on the test set, while good (0.0405 RMSE), is orders of magnitude worse. This large discrepancy indicates that the model has learned the training data "too well" and is unable to generalize this good fit to new, unseen data. It has fit both the signal and the noise. The same pattern, though less evident, is visible in the Armijo variants.

The nature of the Steam dataset is a key factor. Unlike datasets with explicit ratings, the Steam dataset contains implicit feedback in the form of "hours played." "Hours played" is a continuous and noisy variable. A user playing a game for 50.5 hours versus 51 hours does not represent a meaningful difference in preference, but a model trying to minimize MSE will treat this difference seriously. The model overfits by learning these exact, noisy values.

The results for Spectral Accuracy reinforce the conclusions drawn from the Test RMSE.

The Away-Step FW (Armijo) variant, which achieved the best Test RMSE, also reports the best Spectral Accuracy (1.4017). This consistency suggests that the model which best generalizes to unseen data also provides the best approximation of the data's structural properties. The other well-performing models under the Test RMSE metric, such as the Classic FW variants and the Away-Step FW (Diminishing), all cluster around a low spectral accuracy of approximately 1.5. This indicates they are all effective at learning the dominant components of the true data matrix. The Away-Step FW algorithm (Exact LS) exhibit the highest spectral accuracy value, with also a very high Test RMSE value.

7

Conclusions

7.1 BROADER CONSIDERATIONS

The empirical evaluation conducted across three distinct datasets provides a deeper understanding of the practical performance of Frank-Wolfe algorithms for matrix completion. The results reveal that there is no single "best" algorithm; instead, the optimal choice is deeply contingent on the interplay between dataset characteristics, the chosen performance metric (accuracy vs. speed), and the specific algorithmic variant. This aligns with the "No Free Lunch" theorem in optimization, confirming that the theoretical advantages of an algorithm do not always guarantee superior performance in practice.

7.1.1 THE DECISIVE ROLE OF DATASET PROPERTIES

Our experiments clearly demonstrate that dataset characteristics are the primary determinant of algorithmic success.

- **On Standard-Sparsity Datasets (MovieLens):** The MovieLens 100k dataset represents a standard benchmark with explicit ratings and moderate sparsity. The most complex algorithm, **Pairwise FW (Exact LS)**, achieved the best predictive accuracy (Test RMSE of 0.2363). This confirms that for standard recommendation tasks with explicit ratings, the sophisticated atom-swapping mechanism of Pairwise FW is highly effective at finding a high-quality solution. However, this superior accuracy comes at an extreme com-

putational cost (1840 seconds), highlighting the critical trade-off faced when choosing a model.

- **On Dense, Well-Behaved Datasets (Jester):** On the Jester dataset, which is relatively dense and contains explicit, well-structured ratings, the more sophisticated Pairwise FW with an Exact Line Search delivered the highest predictive accuracy (lowest Test RMSE of 0.0265). This is the scenario where the theoretical advantages of advanced variants apply. Pairwise FW, by efficiently swapping atoms within its active set as described in the literature, demonstrates its advantages when gradient information is reliable.
- **On High-Variance, Implicit Datasets (Steam):** The Steam dataset, with its implicit "hours played" feedback, presented a classic case of overfitting. Here, the models using Exact or Armijo line searches learned the training data too well, resulting in a large gap between training and test error. While, the simple **Diminishing step-size** acted as a powerful form of implicit regularization. By underfitting the noisy training data, it achieved a Test RMSE (0.0405) comparable to the best models but at a fraction of the computational cost. This highlights that for noisy, implicit data, preventing overfitting can be more important than achieving perfect convergence on the training set.

A recurring theme across all experiments is the fundamental trade-off between predictive accuracy (Test RMSE), computational efficiency (Time), and algorithmic complexity.

- **High Accuracy at a High Cost:** The **Pairwise FW** variant, particularly with an Exact Line Search, consistently found solutions with the lowest Test RMSE on the Jester and MovieLens datasets. However, it was also, by a large margin, the slowest algorithm. Its sophisticated update rule, while powerful, incurs significant computational cost, making it potentially impractical for very large-scale problems.
- **Maximum Speed at a Cost to Accuracy:** The **Classic FW with a Diminishing step-size** was, without exception, the fastest method across all datasets. Its simplicity and non-adaptive nature make it computationally cheap. However, this speed often came at the cost of suboptimal accuracy, as the pre-determined step-size schedule is incapable of adapting to the local geometry of the problem.
- **The Balanced Contenders:** The **Classic FW with an Armijo or Exact Line Search** consistently offered a strong balance. It provided competitive accuracy, often close to the best-performing models, while being significantly faster than the advanced Away-Step or Pairwise variants. This makes it a reliable baseline for many practical applications.

7.1.2 STEP-SIZE STRATEGY

The choice of step-size strategy proved to be as critical as the choice of the FW variant itself. The adaptive line search methods (Exact and Armijo) generally provided the best accuracy by allowing the algorithm to take optimally-sized steps, aligning with optimization theory. However, the Diminishing step-size demonstrated a powerful, if unintentional, role as a regularizer on the Steam dataset, showcasing that even simple heuristics can be effective in specific contexts.

7.2 FINAL CONCLUSION

Above all, our empirical results confirm that a single, universally optimal algorithm does not exist. The performance of the Frank-Wolfe variants is conditional upon the intrinsic properties of the dataset. In scenarios with well-structured, dense data, the theoretical advantages of advanced methods like **Pairwise FW** materialize as superior predictive performance. Conversely, when faced with sparse or noisy data, these advanced variants can become a liability, and the reliability of the more straightforward **Classic FW** proves to be more effective.

Furthermore, our findings highlight an essential compromise among three competing factors: predictive accuracy, computational speed, and algorithmic sophistication. The Pairwise variant reliably provides the greatest predictive power, yet its high computational cost often makes it impractical. On the other end, the Classic FW with a diminishing step-size is unmatched in speed but generally sacrifices predictive accuracy. In the middle, the **Classic FW utilizing an adaptive line search (Armijo or Exact)** presented itself as a practical option, offering a consistently stable performance across diverse conditions.

Finally, this work makes a strong case for the importance of a comprehensive evaluation. By analyzing convergence dynamics, step-size impacts, and spectral accuracy in addition to the final error, we understood the core reasons for each algorithm’s success or failure. The main takeaway is clear: choosing a recommender algorithm should not be a matter of theory, but of practice. The decision must be based on empirical evidence from tests conducted on the specific data the system will use.

References

- [1] M. Jaggi. Revisiting frank-wolfe: Projection-free sparse convex optimization. [Online]. Available: <https://proceedings.mlr.press/v28/jaggi13.html>
- [2] M. Jaggi and S. Lacoste-Julien. On the global linear convergence of frank-wolfe optimization variants. [Online]. Available: <https://arxiv.org/pdf/1511.05932>
- [3] F. Rinaldi, I. M. Bomze, and D. Zeffiro. Frank–wolfe and friends: a journey into projection-free first-order optimization methods. [Online]. Available: <https://arxiv.org/pdf/2106.10261>